CHAPTER 3

React Native Framework and It’s Components

Content

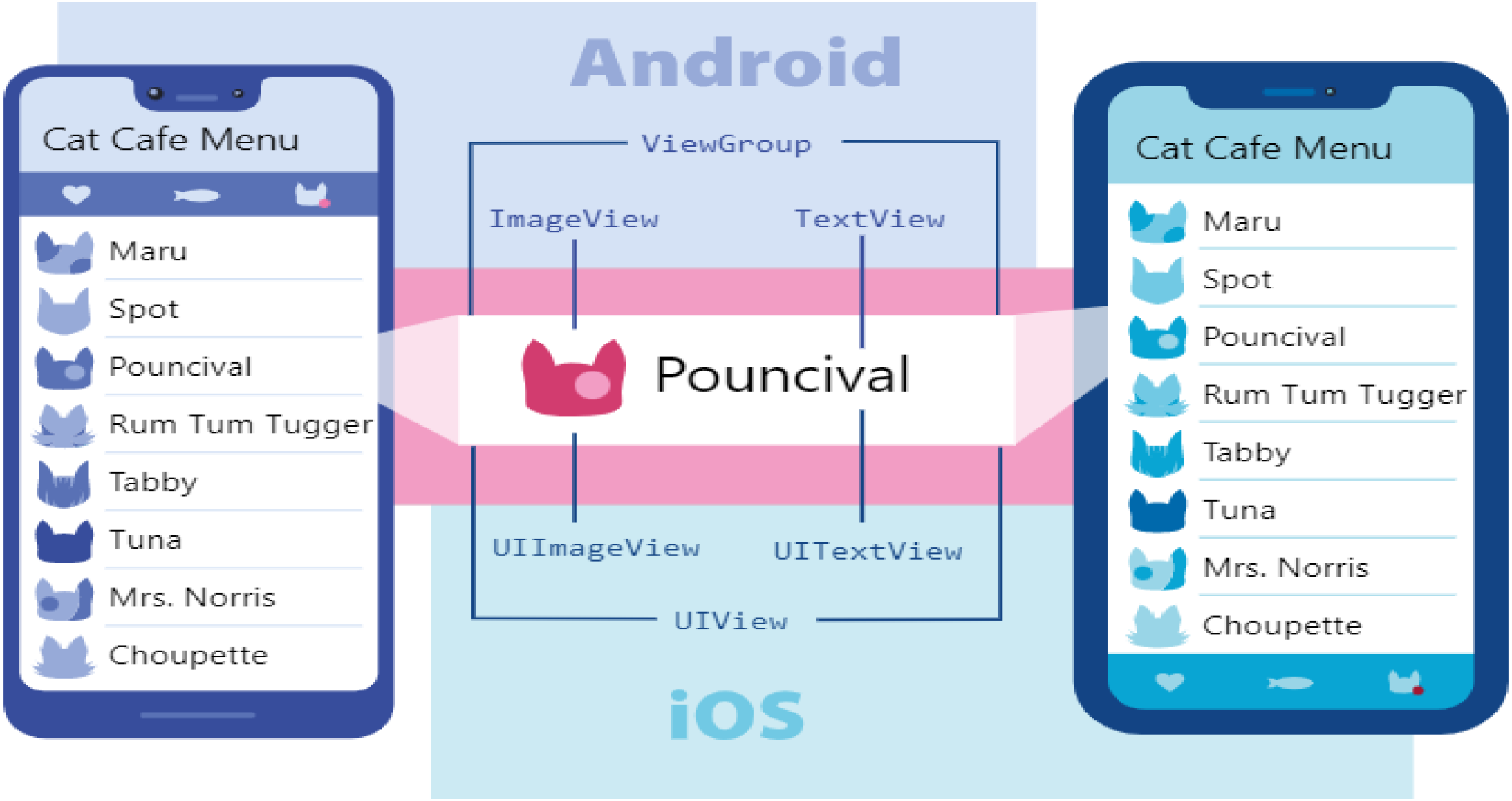
- Overview of React native
- Environmental Setup and First App Hello World
- React native Fundamentals
- React native Styles and layout
- React native Text, TextInput and Switch
- React native Button and Touchable

React Native - Overview

- React Native is an **open source framework** for **building Android and iOS applications** using **React and the app platform's native capabilities**.
- React Native is mainly based on **JavaScript** to access your platform's APIs as well as to describe the appearance and behavior of your UI using React components: bundles of reusable, nestable code.

Basic building block of UI: View

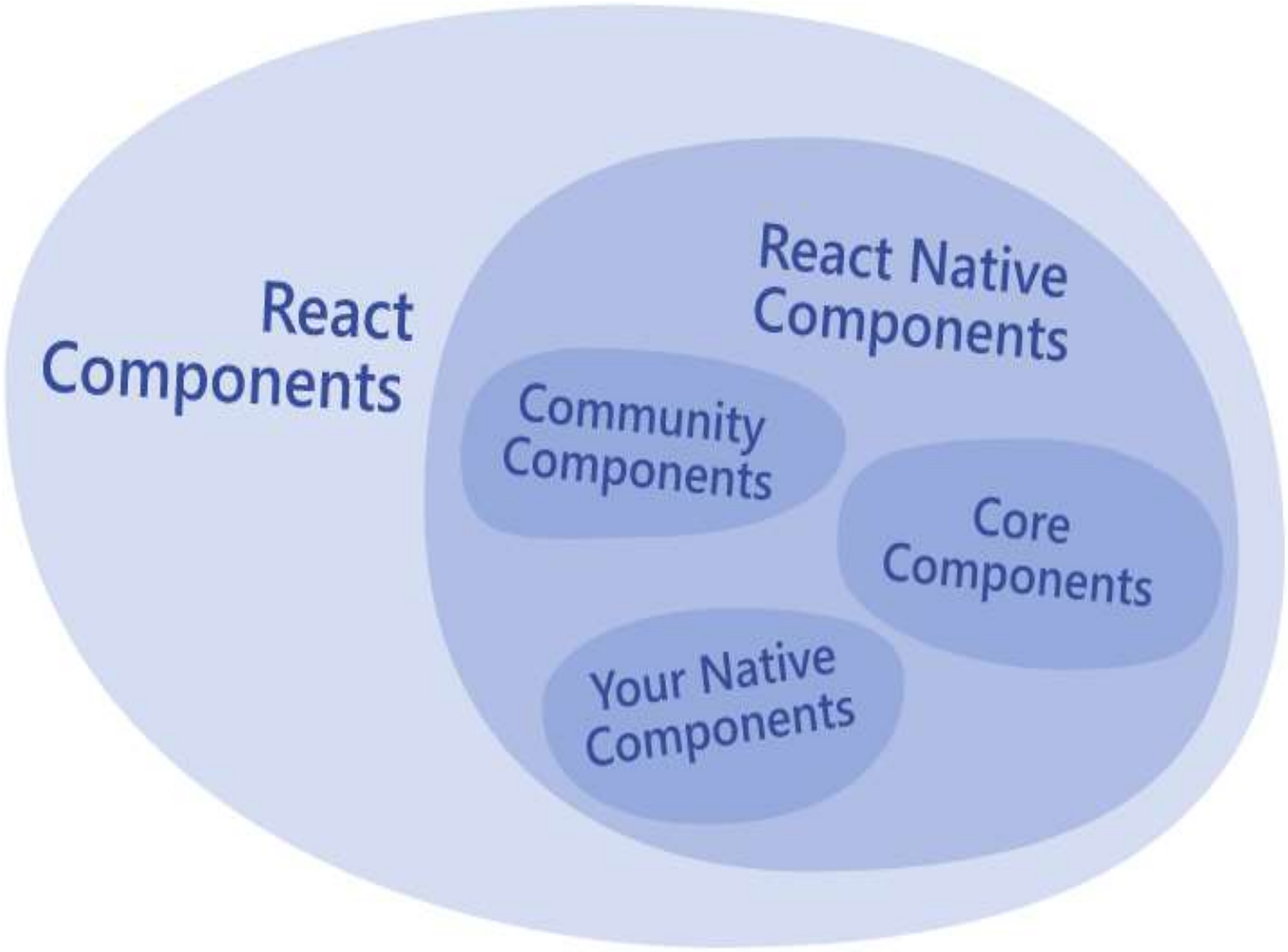
- View is the basic building block of UI in Android and iOS development.
  - It is a **container** that supports layout with **flexbox**, **style**, some touch handling, and **accessibility controls**.
- View maps directly to the **native view equivalent** on whatever platform React Native is running on, whether that is a **UIView**, ** , android.view**, etc.
- Some kinds of views can contain other views.



Just a sampling of the many views used in Android and iOS apps.

React Components vs. React Native Components

- Because React Native **uses the same API structure as React components,** you'll need to understand React component APIs to get started.



React Native vs. Core Component

- **Android** development use **Kotlin or Java** to write views; where as **iOS development** use **Swift** or **Objective-C**.
- With React Native, you can invoke **Views with JavaScript using React components**.
  - Meaning, at runtime, **React Native creates the corresponding Android and iOS views for those components.**

- Because React Native components are backed by the same views as Android and iOS, React Native apps look, feel, and perform like any other apps.
  - This platform-backed components is also called **Native Components.**
    - React Native comes with a set of essential, ready-to-use Native Components you can use to start building your app today. These are **React Native's Core Components**.
    - React Native also lets you build **your own Native Components** for Android and iOS to suit your app’s unique needs.
    - We also have a thriving ecosystem of these **community-contributed components.**

Core Components

- React Native provides a number of built-in Core Components for everything from form controls to activity indicators. These include:
  - Basic Components
  - User Interface
  - List Views
  - Android-specific
  - Others

Basic Components - Most apps end up using these basic components.

View

The most fundamental component for building a UI.

Text

A component for displaying text.

Image

A component for displaying images.

TextInput

A component for inputting text into the app via a keyboard.

ScrollView

Provides a scrolling container that can host multiple components and views.

StyleSheet

Provides an abstraction layer similar to CSS stylesheets.

- **User Interfaces - common user interface controls render on any platform.**

Button	Switch
A basic button component for handling touches that should render nicely on any platform.	Renders a boolean input.
FlatList	**SectionList**
A component for rendering performant scrollable lists.	Like `FlatList`, but for sectioned lists.

- **List Views** - unlike ScrollView (more generic), list view components only render elements that are **currently showing on the screen**. Best to display longlist of data.

- **Android-specific components and API's** - Many of the following components provide wrappers for commonly used Android classes.

BackHandler	DrawerLayoutAndroid
Detect hardware button presses for back navigation.	Renders a `DrawerLayout` on Android.

PermissionsAndroid	ToastAndroid
Provides access to the permissions model introduced in Android M.	Create an Android Toast alert.

Others - These components may be useful for certain applications.

ActivityIndicator

Displays a circular loading indicator.

Alert

Launches an alert dialog with the specified title and message.

Animated

A library for creating fluid, powerful animations that are easy to build and maintain.

Dimensions

Provides an interface for getting device dimensions.

KeyboardAvoidingView

Provides a view that moves out of the way of the virtual keyboard automatically.

Linking

Provides a general interface to interact with both incoming and outgoing app links.

Modal

Provides a simple way to present content above an enclosing view.

PixelRatio

Provides access to the device pixel density.

RefreshControl

This component is used inside a `ScrollView` to add pull to refresh functionality.

StatusBar

Component to control the app status bar.

Common Core Components used

REACT NATIVE UI COMPONENT	ANDROID VIEW	IOS VIEW	WEB ANALOG	DESCRIPTION
`<View>`	`<ViewGroup>`	`<UIView>`	A non-scrollling `<div>`	A container that supports layout with flexbox, style, some touch handling, and accessibility controls
`<Text>`	`<TextView>`	`<UITextView>`	`<p>`	Displays, styles, and nests strings of text and even handles touch events
`<Image>`	`<ImageView>`	`<UIImageView>`	`<img>`	Displays different types of images
`<ScrollView>`	`<ScrollView>`	`<UIScrollView>`	`<div>`	A generic scrolling container that can contain multiple components and views
`<TextInput>`	`<EditText>`	`<UITextField>`	`<input type="text">`	Allows the user to enter text

Running your first react native - app

- Demo - https://reactnative.dev/docs/intro-react

Environmental Setup and First App

- Refer **Lab Session - 4 : Installation procedure**

React Native - Fundamentals

- React Native runs on React, a popular open source library for building user interfaces with JavaScript.
  - So it is important to understand React itself first.
- The following are core concepts behind React and to work with React native:
  - Components
  - JSX
  - props
  - state

Components

- A component is like a blueprints.
  - It can be created as **Function** Component or **Class** Component.
- We focus on Function Component
- Let us define our own function component – HelloReact

cript to i
nents.

```
import React from 'react';
import { Text } from 'react-native';

const HelloReact = () => {};
```

```
import React from 'react';
import { Text } from 'react-native';

const HelloReact = () => {
  return (
    <Text>Hello, I am react native!</Text>
  );
}

export default HelloReact;
```

- Your component starts as function:
- Whatever a function component returns is rendered as a **React element.**
- React elements let you describe what you want to see on the screen.
- So, the HelloReact component will render a <Text> element below.

```
const HelloReact = () => {
  return <Text>Hello, I am react native!</Text>;
};
```

- You can export your function component with JavaScript's export default for use throughout your app like:

```
const HelloReact = () => {
  return <Text>Hello, I am react native!</Text>;
};

export default HelloReact;
```

- Note that there are many ways to export your component. Depending on your app's file structure, you might need to use a different convention.

JSX

- Now take a closer look at that return statement. <Text>Hello, I am your cat!</Text> is using a kind of JavaScript syntax that makes writing elements convenient: JSX.
  - React and React Native use **JSX,** a syntax that lets you write elements inside JavaScript:
    - <Text>Hello, I am react native!</Text>.
- Because JSX is JavaScript, you can use variables inside it. Declaring a name for the HelloReact, name, and embedding it with curly braces inside <Text>.
  - const name = “react";
  - <Text>Hello, I am {name} !</Text>

- Any JavaScript expression will work between curly braces, including function calls:
  - {getFullName(“Lucy", “Afar")}
- The curly braces is like creating a portal into JS functionality in your JSX!
- JSX is included in react library.

```
import React from 'react';
import { Text } from 'react-native';

const getFullName = (firstName, secondName) => {
  return firstName + " " + secondName;
}

const HelloStudent = () => {
  return (
    <Text>
      Hello, I am {getFullName("Lucy", "Afar")}!
    </Text>
  );
}

export default HelloStudent;
```

- React lets you nest components inside each other to create new components.
  - Nestable and reusable components are at the heart of the React paradigm.
    - Text and TextInput are nested inside a View , and React Native will render them together.
- In React Native, View uses Flexbox for its children’s layout.

```
import React from 'react';
import { Text, TextInput, View } from 'react-native';

const Greetings = () => {
  return (
    <View>
      <Text>Hello, I am...</Text>
      <TextInput
        style={{
          height: 40,
          borderColor: 'gray',
          borderWidth: 1
        }}
        defaultValue="Name me!"
      />
    </View>
  );
}

export default Greetings;
```

- Component can be render multiple times and in multiple places without repeating your code by using <Hello/>
  - Any component that renders other components is a **parent component.**
- Here, MultiGreetings is a parent component and each Hello is a **child component.**
  - Each <Hello> renders a unique element—which you can customize with props.

```
import React from 'react';
import { Text, View } from 'react-native';

const Hello = () => {
  return (
    <View>
      <Text>I am also react native!</Text>
    </View>
  );
}

const MultiGreetings = () => {
  return (
    <View>
      <Text>Welcome!</Text>
      <Hello />
      <Hello />
      <Hello />
    </View>
  );
}

export default MultiGreetings;
```

Welcome!
I am also react native!
I am also react native!
I am also react native!

Props – short for “properties”

- **Props** is short for “properties”.
  - Props let you customize React components.
    - E.g. you pass each <Student> a different name for Student to render:

```
import React from 'react';
import { Text, View } from 'react-native';

const Student = (props) => {
  return (
    <View>
      <Text>Hello, I am {props.name}!</Text>

    </View>
  );
}

const Greetings = () => {
  return (
    <View>
      <Student name="Ayalew" />
      <Student name="Jerry" />
      <Student name="Mamo" />
    </View>
  );
}

export default Greetings;
```

- Most of React Native's Core Components can be customized with props, too.
  - E.g. when using Image, you pass it a prop named src to define what image it shows.
- Image has many different props, including style, which accepts a JS object of design and layout related property-value pairs.

```
import React from 'react';
import { Text, View, Image } from 'react-native';

const Display= (props) => {
  return (
    <View>
      <Image
        source={{uri: props.src}}
        style={{width: 200, height: 200}}
      />
      <Text>Hello, I am your cat!</Text>
    </View>
  );
}

const ModImage = () =>{
  return (
    <view>
      <Text>Modifying image source</Text>
      <Display src="https://reactnative.dev/docs/assets/p_cat1.png"/>
    </view>
  );
}

export default ModImage;
```

- **Notice**
  - The double curly braces {{ }} surrounding style‘s width and height.
  - In JSX, JavaScript values are referenced with {}. This is handy if you are passing something other than a string as props, like an array or number: <Student names={["Muna", "Kirubel"]} age={18} />.
  - However, JS objects are also denoted with curly braces: {width: 200, height: 200}. Therefore, to pass a JS object in JSX, you must wrap the object in another pair of curly braces: {{width: 200, height: 200}}
- You can build many things with props and the Core Components (Text, Image, and View). But to build something interactive, you’ll need state.

State

- While you can think of props as arguments you use to configure how components render, state is like a component's personal data storage.
  - State is useful for handling data that changes over time or that comes from user interaction.
  - State gives your components memory!
- **Note** - As a general rule, use props to configure a component when it renders. Use state to keep track of any component data that you expect to change over time.

- You can add state to a component by calling React’s useState Hook.
  - A Hook is a kind of function that lets you “hook into” React features.
    - For example, **useState** is a Hook that lets you add state to function components.

- The example here takes place in a cat cafe where two hungry cats are waiting to be fed.
- Their hunger, which we expect to change over time (unlike their names), is stored as state.
- To feed the cats, press their buttons—which will update their state.

```
import { Button, Text, View } from "react-native";
const Cat = (props) => {
  const [isHungry, setIsHungry] = useState(true);
  return (
    <View>
      <Text>
        I am {props.name}, and I am {isHungry ? "hungry" : "full"}!
      </Text>
      <Button
        onPress={() => { setIsHungry(false);}}
        disabled={!isHungry}
        title={isHungry ? "Pour me some milk, please!" : "Thank you!"}
      />
    </View>
  );
}
const Cafe = () => {
  return (
    <>
      <Cat name="Babycat" />
      <Cat name="Oldcat" />
    </>
  );
}

export default Cafe;
```

- First, you will want to import useState from React.

```
import React, { useState } from 'react';
```

- Then you declare the component's state by calling useState inside its function. In this example, useState creates an isHungry state variable:

```
const Cat = (props) => {
  const [isHungry, setIsHungry] = useState(true);
  // ...
};
```

- Calling **useState** does two things:
  - It creates a “state variable” with an initial value—in this case the state variable is ishungry and its initial value is true
  - It creates a function to set that state variable’s value—setishungry
  - It doesn’t matter what names you use. But it can be handy to think of the pattern as
    - [<getter>, <setter>] = useState(<initialValue>).

- Next you add the Button Core Component and give it an onPress prop:
- Now, when someone presses the button, onPress will fire, calling the setIsHungry(false). This sets the state variable isHungry to false.
- When isHungry is false, the Button's disabled prop is set to true and its title also changes:
- Although isHungry is a const, it is seemingly reassignable! What is happening is when a state-setting function like setIsHungry is called, its component will re-render.

```
<Button
  onPress={() => {
    setIsHungry(false);
  }}
  //..
/>
```

```
<Button
  //..
  disabled={!isHungry}
  title={isHungry ? 'Pour me some milk, please!' : 'Thank you!'}
/>
```

- In this case the Cat function will run again—and this time, useState will give us the next value of isHungry.
- Finally, put your cats inside a Cafe component:
- See the <> and </>. These bits of JSX are fragments. Adjacent JSX elements must be wrapped in an enclosing tag.
- Fragments let you do that without nesting an extra, unnecessary wrapping element like View.

```
const Cafe = () => {
  return (
    <>
      <Cat name="Munkustrap" />
      <Cat name="Spot" />
    </>
  );
};
```

React-native Styling and Layout

- React Native style your application using JavaScript.
  - All core components accept a prop named **style**.
  - The style names and values usually match how CSS works on the web, except names are written using **camel casing**, e.g. **backgroundColor** rather than ~~background-color~~.
- The style prop can be a plain old JavaScript object. That's what we usually use for example code. You can also pass an array of styles - the last style in the array has precedence, so you can use this to inherit styles.

- As a component grows in complexity, it is often cleaner to use **StyleSheet.create** to define several styles in one place.
- One common pattern is to make your component accept a style prop which in turn is used to style subcomponents. You can use this to make styles "cascade" the way they do in CSS.
- There are a lot more ways to customize the text style.

```
import React from 'react';
import { StyleSheet, Text, View } from 'react-native';

const LotsOfStyles = () => {
    return (
      <View style={styles.container}>
        <Text style={styles.red}>just red</Text>
        <Text style={styles.bigBlue}>just bigBlue</Text>
        <Text style={[styles.bigBlue, styles.red]}>bigBlue, then red</Text>
        <Text style={[styles.red, styles.bigBlue]}>red, then bigBlue</Text>
      </View>
    );
};

const styles = StyleSheet.create({
  container: {     marginTop: 50, },
  bigBlue: {
    color: 'blue', fontWeight: 'bold', fontSize: 30, },
  red: {    color: 'red',
  },
});

export default LotsOfStyles;
```

Styling - Height and Width

- A component's height and width determine its size on the screen.
- This can be done in three ways:
  - Fixed Dimensions,
  - Flex Dimensions
  - Percentage Dimensions

Fixed Dimensions

- The general way to set the dimensions of a component is by adding a fixed width and height to style.
  - All dimensions in React Native are unitless, and represent **density-independent pixels**.
- Setting dimensions this way is common for components whose size should always be fixed to a number of points and not calculated based on screen size.

```
import React from 'react';
import { View } from 'react-native';

const FixedDimensionsBasics = () => {
  return (
    <View>
      <View style={{
        width: 50, height: 50, backgroundColor: 'powderblue'
      }} />
      <View style={{
        width: 100, height: 100, backgroundColor: 'skyblue'
      }} />
      <View style={{
        width: 150, height: 150, backgroundColor: 'steelblue'
      }} />
    </View>
  );
};

export default FixedDimensionsBasics;
```

Flex Dimensions

- Use flex in a component's style to have the component expand and shrink dynamically based on available space.
- Use flex: 1, which tells a component to fill all available space, shared evenly amongst other components with the same parent.
    - The larger the flex given, the higher the ratio of space a component will take compared to its siblings.
- **Note** - A component can only expand to fill available space if its parent has dimensions greater than 0.
    - If a parent does not have either a fixed width and height or flex, the parent will have dimensions of 0 and the flex children will not be visible.

```
import React from 'react';
import { View } from 'react-native';

const FlexDimensionsBasics = () => {
  return (
    // Try removing the `flex: 1` on the parent View.
    // The parent will not have dimensions, so the children can't expand.
    // What if you add `height: 300` instead of `flex: 1`?
    <View style={{ fllex: 1  }}>
      <View style={{ height: 100, backgroundColor: 'powderblue' }} />
      <View style={{ backgroundColor: 'skyblue' }} />
      <View style={{ flex: 3, backgroundColor: 'steelblue' }} />
    </View>
  );
};

export default FlexDimensionsBasics;
```

Percentage Dimensions

- If you want to fill a certain portion of the screen, but you don't want to use the flex layout,
  - use percentage values in the component's style.
- Similar to flex dimensions, percentage dimensions require parent with a defined size.

```react-native-snack
import React from 'react';
import { View } from 'react-native';

const PercentageDimensionsBasics = () => {
  // Try removing the `height: '100%'` on the parent View.
  // The parent will not have dimensions, so the children can't expand.
  return (
    <View style={{ height: '100%' }}>
      <View style={{
        height: '15%', backgroundColor: 'powderblue'
      }} />
      <View style={{
        width: '66%', height: '15%', backgroundColor: 'skyblue'
      }} />
      <View style={{
        width: '33%', height: '50%', backgroundColor: 'steelblue'
      }} />
    </View>
  );
};

export default PercentageDimensionsBasics;
```

Layout with Flexbox

- A component can specify the layout of its children using the Flexbox algorithm. Flexbox is designed to provide a consistent layout on different screen sizes.
- You will normally use a combination of **flexDirection**, **alignItems**, and **justifyContent** to achieve the right layout.

Flex

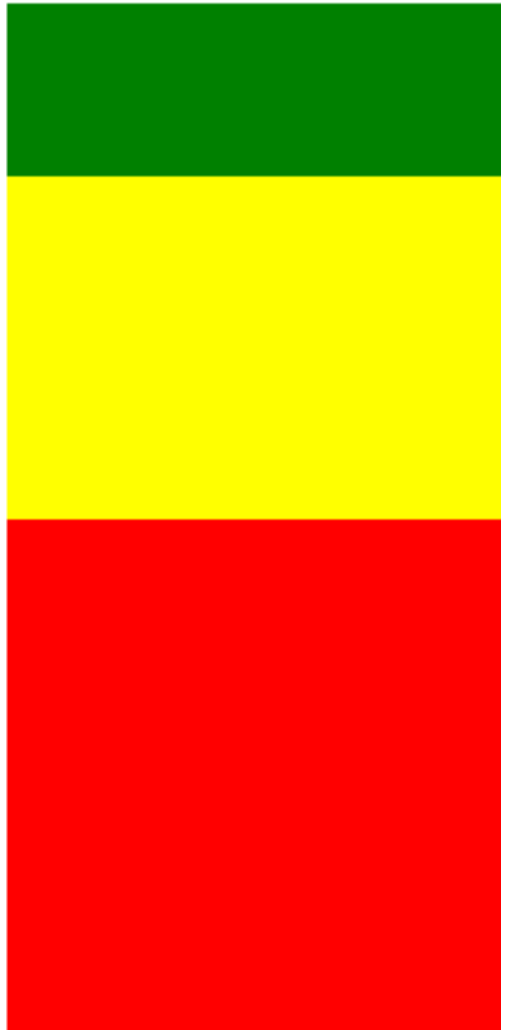
- flex will define how your items are going to “fill” over the available space along your main axis.
- Space will be divided according to each element's flex property.
  - In the following example, the red, yellow, and green views are all children in the container view that has flex: 1 set. The green view uses flex: 1 , the yellow view uses flex: 2, and the red view uses flex: 3 . 1+2+3 = 6, which means that the green view will get 1/6 of the space, the yellow 2/6 of the space, and the red 3/6 of the space.
- See the code and the output in the next slid.

```react-native
import React from "react";
import { StyleSheet, Text, View } from "react-native";

const Flex = () => {
  return (
    <View style={[styles.container, {
      // Try setting `flexDirection` to `"row"`.
      flexDirection: "column"
    }]}>
      <View style={{ flex: 1, backgroundColor: "green" }} />
      <View style={{ flex: 2, backgroundColor: "yellow" }} />
      <View style={{ flex: 3, backgroundColor: "red" }} />
    </View>
  );
};

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
  },
});

export default Flex;
```



Flex Direction

- **flexDirection** controls the direction in which the children of a node are laid out. This is also referred to as the **main axis**. The **cross axis** is the axis perpendicular to the main axis, or the axis which the wrapping lines are laid out in.
  - **column** (default value) Align children from top to bottom. If wrapping is enabled, then the next line will start to the right of the first item on the top of the container.
  - **row** Align children from left to right. If wrapping is enabled, then the next line will start under the first item on the left of the container.
  - **column-reverse** Align children from bottom to top. If wrapping is enabled, then the next line will start to the right of the first item on the bottom of the container.
  - **row-reverse** Align children from right to left. If wrapping is enabled, then the next line will start under the first item on the right of the container.

Flex Direction

Expo

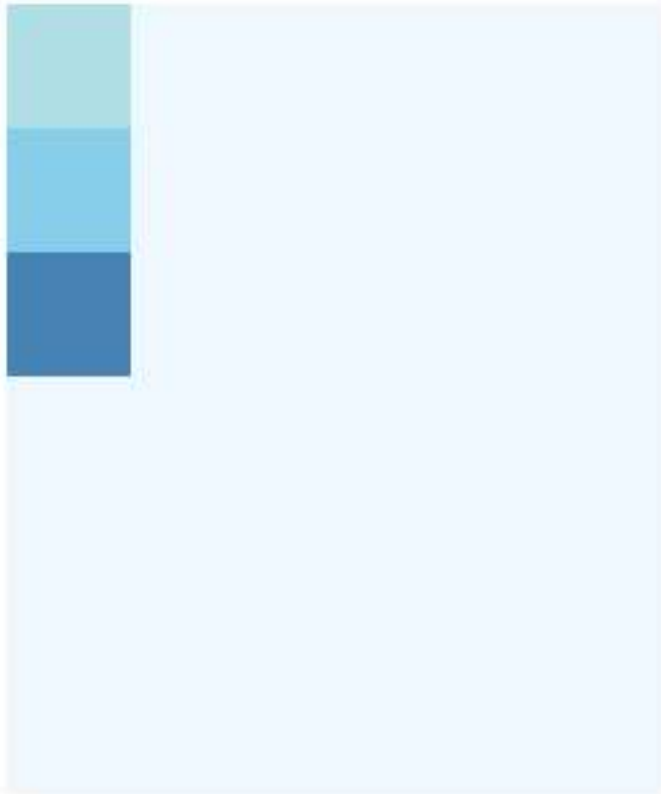
```
import React, { useState } from "react";
import { StyleSheet, Text, TouchableOpacity, View } from "react-native";

const FlexDirectionBasics = () => {
  const [flexDirection, setflexDirection] = useState("column");

  return (
    <PreviewLayout
      label="flexDirection"
      values={["column", "row", "row-reverse", "column-reverse"]}
      selectedValue={flexDirection}
      setSelectedValue={setflexDirection}
    >
      <View
        style={[styles.box, { backgroundColor: "powderblue" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "skyblue" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "steelblue" }]}
      />
    </PreviewLayout>
  );
};
```

flexDirection

column | row | row-reverse | column-reverse



Preview

My Device | iOS | Android | Web

Layout Direction

- Layout direction specifies the direction in which children and text in a hierarchy should be laid out. Layout direction also affects what edge start and end refer to.
- By default, React Native lays out with LTR layout direction. In this mode start refers to left and end refers to right.
  - **LTR** (default value) Text and children are laid out from left to right. Margin and padding applied to the start of an element are applied on the left side.
  - **RTL** Text and children are laid out from right to left. Margin and padding applied to the start of an element are applied on the right side.

Flex Direction

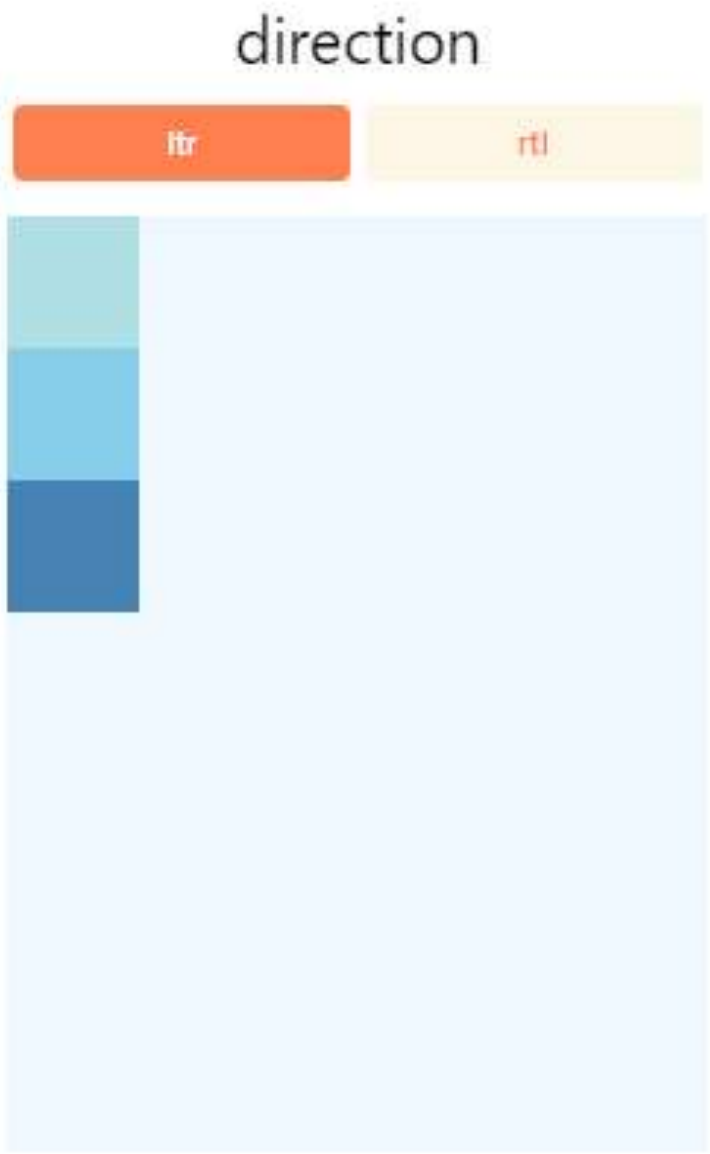
Expo

```react-native-example
import React, { useState } from "react";
import { View, TouchableOpacity, Text, StyleSheet } from "react-native";

const DirectionLayout = () => {
  const [direction, setDirection] = useState("ltr");

  return (
    <PreviewLayout
      label="direction"
      selectedValue={direction}
      values={["ltr", "rtl"]}
      setSelectedValue={setDirection}>
      <View
        style={[styles.box, { backgroundColor: "powderblue" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "skyblue" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "steelblue" }]}
      />
    </PreviewLayout>
  );
};

const PreviewLayout = ({
```



Justify Content

- **justifyContent** describes how to align children within the main axis of their container.
  - **flex-start**(default value) Align children of a container to the start of the container's main axis.
  - **flex-end** Align children of a container to the end of the container's main axis.
  - **center** Align children of a container in the center of the container's main axis.
  - **space-between** Evenly space off children across the container's main axis, distributing the remaining space between the children.
  - **space-around** Evenly space off children across the container's main axis, distributing the remaining space around the children. Compared to space-between, using space-around will result in space being distributed to the beginning of the first child and end of the last child.
  - **space-evenly** Evenly distribute children within the alignment container along the main axis. The spacing between each pair of adjacent items, the main-start edge and the first item, and the main-end edge and the last item, are all exactly the same.

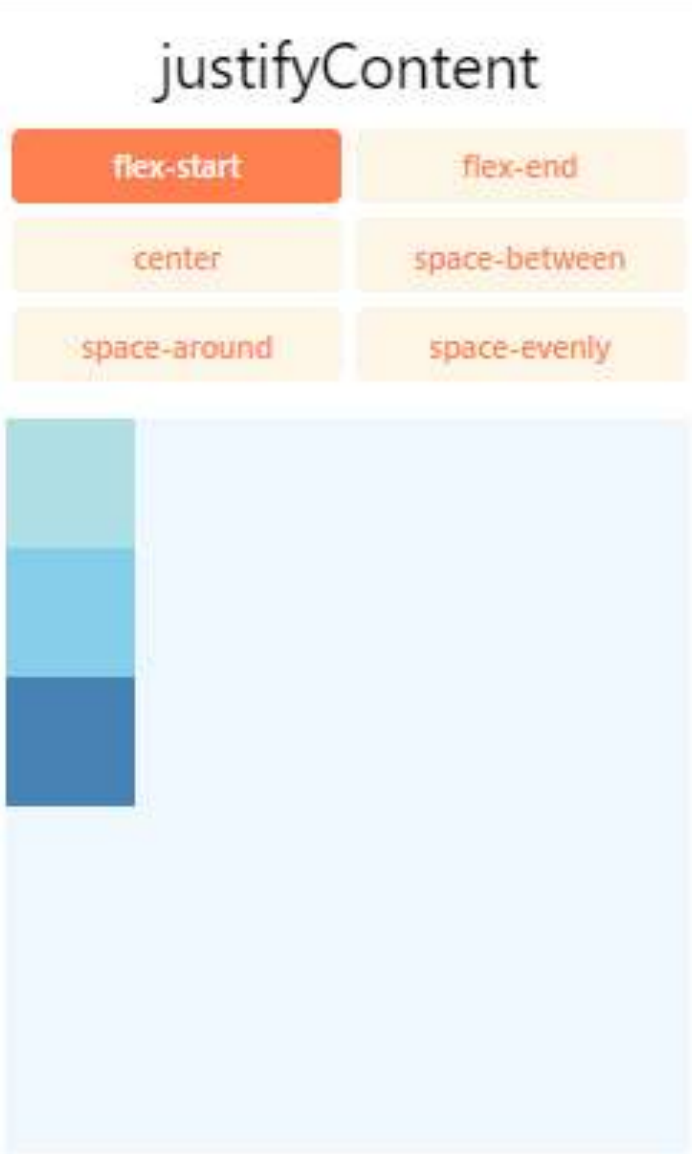
Justify Content

Expo

```
import React, { useState } from "react";
import { View, TouchableOpacity, Text, StyleSheet } from "react-native";

const JustifyContentBasics = () => {
  const [justifyContent, setJustifyContent] = useState("flex-start");

  return (
    <PreviewLayout
      label="justifyContent"
      selectedValue={justifyContent}
      values={[
        "flex-start",
        "flex-end",
        "center",
        "space-between",
        "space-around",
        "space-evenly",
      ]}
      setSelectedValue={setJustifyContent}
    >
      <View
        style={[styles.box, { backgroundColor: "powderblue" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "skyblue" }]}
      />
```



Align Items

- **alignItems** describes how to align children along the cross axis of their container. It is very similar to justifyContent but instead of applying to the main axis, alignItems applies to the cross axis.
  - **stretch** (default value) Stretch children of a container to match the height of the container's cross axis.
  - **flex-start** Align children of a container to the start of the container's cross axis.
  - **flex-end** Align children of a container to the end of the container's cross axis.
  - **center** Align children of a container in the center of the container's cross axis.
  - **baseline** Align children of a container along a common baseline. Individual children can be set to be the reference baseline for their parents.

Align Items

Expo

```
import React, { useState } from "react";
import {
  View,
  TouchableOpacity,
  Text,
  StyleSheet,
} from "react-native";

const AlignItemsLayout = () => {
  const [alignItems, setAlignItems] = useState("stretch");

  return (
    <PreviewLayout
      label="alignItems"
      selectedValue={alignItems}
      values={[
        "stretch",
        "flex-start",
        "flex-end",
        "center",
        "baseline",
      ]}
      setSelectedValue={setAlignItems}
    >
```

Align Content

- **alignContent** defines the distribution of lines along the cross-axis. This only has effect when items are wrapped to multiple lines using flexWrap.
  - **flex-start** (default value) Align wrapped lines to the start of the container's cross axis.
  - **flex-end** Align wrapped lines to the end of the container's cross axis.
  - **stretch** (default value when using Yoga on the web) Stretch wrapped lines to match the height of the container's cross axis.
  - **center** Align wrapped lines in the center of the container's cross axis.
  - **space-between** Evenly space wrapped lines across the container's cross axis, distributing the remaining space between the lines.
  - **space-around** Evenly space wrapped lines across the container's cross axis, distributing the remaining space around the lines. Compared to space-between, using space-around will result in space being distributed to the beginning of the first line and the end of the last line.

Align Content

Expo

```
import React, { useState } from "react";
import { View, TouchableOpacity, Text, StyleSheet } from "react-native";

const AlignContentLayout = () => {
  const [alignContent, setAlignContent] = useState("flex-start");

  return (
    <PreviewLayout
      label="alignContent"
      selectedValue={alignContent}
      values={[
        "flex-start",
        "flex-end",
        "stretch",
        "center",
        "space-between",
        "space-around",
      ]}
      setSelectedValue={setAlignContent}>
      <View
        style={[styles.box, { backgroundColor: "orangered" }]}
      />
      <View
        style={[styles.box, { backgroundColor: "orange" }]}
```

alignContent

flex-start | flex-end

stretch | center

space-between | space-around

Flex Wrap

- The **flexWrap** property is set on containers and it controls what happens when children overflow the size of the container along the main axis.
  - By default, children are forced into a single line (which can shrink elements). If wrapping is allowed, items are wrapped into multiple lines along the main axis if needed.
- When wrapping lines, **alignContent** can be used to specify how the lines are placed in the container.

Flex Basis, Grow, and Shrink

- **flexBasis** is an axis-independent way of providing the default size of an item along the main axis. Setting the flexBasis of a child is similar to setting the width of that child if its parent is a container with flexDirection: row or setting the height of a child if its parent is a container with flexDirection: column.
  - The flexBasis of an item is the default size of that item, the size of the item before any flexGrow and flexShrink calculations are performed.

- **flexGrow** describes how any space within a container should be distributed among its children along the main axis. After laying out its children, a container will distribute any remaining space according to the flex grow values specified by its children.
  - flexGrow accepts any floating point value >= 0, with 0 being the default value. A container will distribute any remaining space among its children weighted by the children’s flexGrow values.

- **flexShrink** describes how to shrink children along the main axis in the case in which the total size of the children overflows the size of the container on the main axis.
  - flexShrink is very similar to flexGrow and can be thought of in the same way if any overflowing size is considered to be negative remaining space.
    - These two properties also work well together by allowing children to grow and shrink as needed.
  - flexShrink accepts any floating point value >= 0, with 0 being the default value (on the web, the default is 1). A container will shrink its children weighted by the children's flexShrink values.

Width and Height

- The width property specifies the width of an element's content area. Similarly, the height property specifies the height of an element's content area.
- Both width and height can take the following values:
  - **auto** (default value) React Native calculates the width/height for the element based on its content, whether that is other children, text, or an image.
  - **pixels** Defines the width/height in absolute pixels. Depending on other styles set on the component, this may or may not be the final dimension of the node.
  - **percentage** Defines the width or height in percentage of its parent's width or height, respectively.

Absolute & Relative Layout

- The position type of an element defines how it is positioned within its parent.
  - **relative** (default value) By default, an element is positioned relatively. This means an element is positioned according to the normal flow of the layout, and then offset relative to that position based on the values of top, right, bottom, and left. The offset does not affect the position of any sibling or parent elements.
  - **absolute** When positioned absolutely, an element doesn't take part in the normal layout flow. It is instead laid out independent of its siblings. The position is determined based on the top, right, bottom, and left values.

Absolute

Expo

```
import React, { useState } from "react";
import {
  View,
  SafeAreaView,
  TouchableOpacity,
  Text,
  StyleSheet,
} from "react-native";

const PositionLayout = () => {
  const [position, setPosition] = useState("relative");

  return (
    <PreviewLayout
      label="position"
      selectedValue={position}
      values={["relative", "absolute"]}
      setSelectedValue={setPosition}
    >
      <View
        style={[
          styles.box,
          {
            top: 25,
            left: 25,
```

position

relative absolute

Core Component - Text

- A React component for **displaying text** which supports **nesting**, **styling**, and **touch handling**.
- This component can be nested and it can inherit properties from parent to child.
- **Text props**
  - **numberOfLines:** Used to truncate the text with an ellipsis after computing the text layout, including line wrapping, such that the total number of lines does not exceed this number.
    - Setting this property to 0 will result in unsetting this value, which means that no lines restriction will be applied.
  - **onPress**: This function is called on press

- **Import**
  - **import {Text } from 'react-native‘**
- **Syntax:**

  <Text> . . . </Text>
- **Text Styles**
  - color, fontFamily, fontSize, fontStyle, fontWeight, letterSpacing, lineHeight, textAlign, textTransform
- **Example**
  - Design 3 Text components with the following texts

  I ‘m bold

  I’m italic

  I’m underline

```
<View>
    <Text style={styles.bold}>I'm bold!</Text>
    <Text style={styles.italic}>I'm italic!</Text>
    <Text style={styles.underline}>I'm underlined!</Text>
</View>
```

```
const styles = StyleSheet.create({
    bold: {fontWeight: 'bold'},
    italic: {fontStyle: 'italic'},
    underline: {textDecorationLine: 'underline'}
})
```

Core Component - TextInput

- A foundational component for **inputting text** into the app via a keyboard.
  - Props provide configurability for several features, such as **auto-correction**, **auto-capitalization**, **placeholder text**, and **different keyboard types**, such as a numeric keypad.
- The most basic use case is to plop down a TextInput and subscribe to the **onChangeText** events to read the user input.
- There are also other events, such as **onSubmitEditing** and **onFocus** that can be subscribed to.

- **Import**
  - **import { TextInput } from 'react-native‘**
- **Syntax:**

  <TextInput />

- **TextInput Styles**
  - borderWidth, borderColor, borderRadius, padding, margin, width, backgroundColor,
- **TextInput props**
  - **multiline**: the TextInput component can also accept multi-line text. This can be useful for cases like acquiring user feedback:

- **placeholder**: the string that will be rendered before text input has been entered.
- **maxLength** : Want to restrict the user's input length? This can be possible with the maxLength prop.
- **editable** : you can block input to your element using the editable prop.
- **keyboardType** : To change keyboard types [default, number-pad, decimal-pad, numeric, email-address, phone-pad]
- **secureTextEntry** : enable us to customize our text field so that the entered data is obscured. [Password filed]
- **onChangeText:** capture the data every time the input's value changes.
- **onSubmitEditing:** save data only if the user submits the form.

```
<TextInput
    style={styles.input}
    placeholder={"Enter your email"}
    multiline={false}
    maxLength={10}
    keyboardType={'email-address'}
/>
```

```
const styles = StyleSheet.create({
    input:{
        borderWidth:2,
        borderColor:'#ddd',
        width:300,
        padding:10,
        margin:10,
    },
});
```

Core Component - Switch

- Renders a boolean input.
- This is a controlled component that requires an **onValueChange** callback that updates the value prop in order for the component to reflect user actions.
  - If the value prop is not updated, the component will continue to render the supplied value prop instead of the expected result of any user actions.

- **Import**
  - **import { Switch} from 'react-native‘**
- **Syntax:**

  <Switch/>

- **Switch Styles**
  - -
- **Switch props**
  - **onValueChange:** Invoked when the user tries to change the value of the switch. Receives the new value as an argument. [Option: use onChange]

- **thumbColor**: Color of the foreground switch grip. If this is set on iOS, the switch grip will lose its drop shadow.
- **trackColor**: Custom colors for the switch track.
  - iOS: When the switch value is false, the track shrinks into the border. If you want to change the color of the background exposed by the shrunken track, use **ios_backgroundColor**.
- **value**: The value of the switch. If true the switch will be turned on. Default value is false.

```
import React, { Component } from 'react';
import {View, Text, Switch,StyleSheet} from 'react-native';
export default class MySwicth extends Component {
    state = {
        switchValue: false
    };
  render() {
    return (
      <View>
        <Text style={styles.textStyle}>{this.state.switchValue ? 'on' :'off'}</Text>
        <Switch
            styles={styles.input}
            trackColor={{ false: "#767577", true: "#81b0ff" }}
            thumbColor={this.state.switchValue ? "#f5dd4b" : "#f4f3f4"}
            ios_backgroundColor="#3e3e3e"
            onValueChange ={(switchValue)=>this.setState({switchValue})}
            value={this.state.switchValue}
        />
      </View>
    )
  }
}
```

React Native Buttons

- Facebook offers the Button component, which can be used as a generic button.
- **Import**
  - **import { Button } from 'react-native‘**
- **Syntax**
  - **<Button/>**

- **Button props**
  - **title:** Text to display inside the button. On Android the given title will be converted to the uppercased form.
  - **onPress:** Handler to be called when the user taps the button.
  - **color**: Color of the text (iOS), or background color of the button (Android).
  - **disabled**: If true, disable all interactions for this component.

```
export default class MyButton extends Component {
  render() {
    return (
      <View>
        <Button
          title="Login"
          color="red"
          disabled={false}
          onPress={()=> alert("Button pressed!")}
        />
      </View>
    );
  }
}
```

- If the default Button component does not suit your needs, you can use one of the following components instead.
  - **Touchable Opacity**
    - This element will change the opacity of an element when touched.
  - **Touchable Highlight**
    - When a user presses the element, it will get darker and the underlying color will show through.
  - **Touchable Native Feedback**
    - This will simulate ink animation when the element is pressed.
  - **Touchable Without Feedback**
    - This should be used when you want to handle the touch event without any animation. Usually, this component is not used much.

TouchableOpacity

```
import React from 'react'
import { TouchableOpacity, StyleSheet, View, Text } from 'react-native'

const App = () => {
   return (
      <View style = {styles.container}>
         <TouchableOpacity>
            <Text style = {styles.text}>
               Button
            </Text>
         </TouchableOpacity>
      </View>
   )
}
```

Touchable Highlight

```
import React from 'react'
import { View, TouchableHighlight, Text, StyleSheet } from 'react-native'

const App = (props) => {
   return (
      <View style = {styles.container}>
         <TouchableHighlight>
            <Text style = {styles.text}>
               Button
            </Text>

         </TouchableHighlight>
      </View>
   )
}
```

Touchable Native Feedback

```
import React from 'react'
import { View, TouchableNativeFeedback, Text, StyleSheet } from 'react-native'

const Home = (props) => {
   return (
      <View style = {styles.container}>
         <TouchableNativeFeedback>
            <Text style = {styles.text}>
               Button
            </Text>
         </TouchableNativeFeedback>
      </View>
   )
}
```

Touchable Without Feedback

```
<TouchableWithoutFeedback>
    <Text>
        Button
    </Text>
</TouchableWithoutFeedback>
```

Thank You

Questions?